CIS 1904: Haskell

Monads

Logistics

- HW 7 due yesterday
- HW 8 will be released tomorrow

What is a pure language?

- No side effects:
  - Printing
  - Reading from memory
  - Anything except evaluating a term down to a simpler term
    - e.g. $2 + 2 + 2 \rightarrow 4 + 2 \rightarrow 6$
- Easy to reason about
- Helps avoid classes of bugs (esp. concurrency bugs)
- We will see later how Haskell manages this
  - (It's monads)

Monads

- A construct for modelling effectful code, with functions for *sequencing* actions
- A typeclass for type constructors of kind `* -> *`
- An expansion of `Applicative`

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b
```

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b
```

```
Just (div 3) <*> Just 0
```

What happens when we run this code?

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b
```

```
Just (div 3) <*> Just 0
```

It crashes! We can use `<*>` to pass along the information that either subcomputation fails, but not that *the combination of their results* fails.

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b
```

```
Just (safeDiv 3) <*> Just 0
```

What about `safeDiv`?

```
safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv x y = Just (div x y)
```

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b
```

What about `safeDiv`?

```
safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv x y = Just (div x y)
```

What is the problem with this code?

```
Just (safeDiv 3) <*> Just 0
```

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

What about `safeDiv`?

```
safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv x y = Just (div x y)
```

What is the problem with this code?

```
Just (safeDiv 3) <*> Just 0
```

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b

    foo :: (a -> f b) -> f a -> f b
```

```
Just (safeDiv 3) <*> Just 0
```

```
safeDiv 3 `foo` Just 0
```

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b

    foo :: (a -> f b) -> f a -> f b
```

Using `foo`, write a function `safeHeadMaybe` that, given a `Maybe [a]`, safely gets the head of its input without pattern matching directly.

(We have not defined `foo` yet, so you will not be able to test it.)

Is this doable with `<*>`?

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    (<*>) :: f (a -> b) -> f a -> f b

    foo :: (a -> f b) -> f a -> f b
```

Does `foo` subsume `<*>`, i.e., can we implement `<*>` using `foo`?

Applicative

```
foo :: (a -> f b) -> f a -> f b
```

```
(<*>) :: f (a -> b) -> f a -> f b
(<*>) g x = foo (\y -> foo (\h -> pure (h y)) g) x
```

Applicative

```
class Functor f => Applicative f where
    pure :: a -> f a

    foo :: (a -> f b) -> f a -> f b
```

Applicative

```
class Functor f => Monad f where
    pure :: a -> f a

    foo :: (a -> f b) -> f a -> f b
```

Applicative

```
class Applicative f => Monad f where
    pure :: a -> f a

    foo :: (a -> f b) -> f a -> f b
```

Applicative

```
class Applicative f => Monad f where
    return :: a -> f a

    foo :: (a -> f b) -> f a -> f b
```

Applicative

```
class Applicative f => Monad f where
    return :: a -> f a

    (>>=) :: (a -> f b) -> f a -> f b
```

Applicative

```
class Applicative f => Monad f where
    return :: a -> f a

    (>>=) :: f a -> (a -> f b) -> f b
```

Monad

```
class Applicative m => Monad m where
    return :: a -> m a

    (>>=) :: m a -> (a -> m b) -> m b
```

Monads

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Note: `return` is NOT the same concept as a return statement in other languages. It is the same idea as `pure`; we are “putting” a value “in” the monad.

Monads

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Note: `return` is NOT the same concept as a return statement in other languages. It is the same idea as `pure`; we are “putting” a value “in” the monad.

Monads model *effects*.
`return v` creates a representation of `v` in our effect model.

Monads

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Note: `return` is NOT the same concept as a return statement in other languages. It is the same idea as `pure`; we are “putting” a value “in” the monad.

Monads model *effects*.
`(>>=)` (“bind”) *sequences* effects.

Monads

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Note: `return` is NOT the same concept as a return statement in other languages. It is the same idea as `pure`; we are “putting” a value “in” the monad.

Monads model *effects*.
`(>>=)` (“bind”) *sequences* effects.
An `m a` is an effectful computation that could produce an `a`.
A `a -> m b` uses that `a` and creates an `m b`, an effectful computation that could produce a `b`.

Maybe

```
safeDiv :: Int -> Int -> Maybe Int
safeDiv _ 0 = Nothing
safeDiv x y = Just (div x y)
```

The effect we are modelling here is program failure, e.g., from a divide by 0.

Maybe

```
instance  Monad Maybe  where
    return :: a -> Maybe a
    return x =
```

The effect we are modelling here is program failure, e.g., from a divide by 0.

Maybe

```
instance  Monad Maybe  where
    return :: a -> Maybe a
    return x = Just x
```

The effect we are modelling here is program failure, e.g., from a divide by 0.

Maybe

```
instance  Monad Maybe  where
    return :: a -> Maybe a
    return x = Just x

    (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
```

The effect we are modelling here is program failure, e.g., from a divide by 0.

Maybe

```
instance  Monad Maybe  where
    return :: a -> Maybe a
    return x = Just x

    (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
    Nothing  >>= _  = Nothing
```

The effect we are modelling here is program failure, e.g., from a divide by 0.

Maybe

```
instance  Monad Maybe  where
    return :: a -> Maybe a
    return x = Just x

    (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
    Nothing  >>= _  = Nothing
    (Just x) >>= f  = f x
```

The effect we are modelling here is program failure, e.g., from a divide by 0.

Maybe

```
divAdd :: Int -> Int -> Int -> Int -> Maybe Int
divAdd x1 x2 x3 x4 = case safeDiv x1 x2 of
    Nothing -> Nothing
    Just r1 -> case safeDiv x3 x4 of
        Nothing -> Nothing
        Just r2 -> Just (r1 + r2)
```

Maybe

```
divAdd :: Int -> Int -> Int -> Int -> Maybe Int
divAdd x1 x2 x3 x4 = case safeDiv x1 x2 of
    Nothing -> Nothing
    Just r1 -> case safeDiv x3 x4 of
        Nothing -> Nothing
        Just r2 -> Just (r1 + r2)

divAdd’ x1 x2 x3 x4 =
    safeDiv x1 x2 >>=
    \r1 -> (safeDiv x3 x4 >>=
        \r2 -> return (r1 + r2)
```

Do Notation

```
foo f x = x >>= (\v -> return (f v))
```

Do Notation

```
foo f x = x >>= (\v -> return (f v))
```

```
foo f x = do
    v <- x
    return (f v)
```

Do Notation

```
foo f x = x >>= (\v -> return (f v))
```

```
foo f x = do
    v <- x
    return (f v)
```

do signals that we are in this special notational block.

Do Notation

```
foo f x = x >>= (\v -> return (f v))
```

```
foo f x = do
    v <- x
    return (f v)
```

We do not need the keyword `do` unless we are using do notation.

Do Notation

```
foo f x = x >>= (\v -> return (f v))
```

```
foo f x = do
    v <- x
    return (f v)
```

The backward arrow pulls out the result of a computation.

Do Notation

```
foo f x = x >>= (\v -> return (f v))
```

```
foo f x = do
    v <- x
    return (f v)
```

The last line must have a monadic type (e.g., `Maybe Int`).

Do Notation

```
foo f x = x >>= (\v -> f v)
```

```
foo f x = do
    v <- x
    f v
```

The last line must have a monadic type (e.g., `Maybe Int`).

Do Notation

```
foo f x = x >>= (\v -> f v)
```

```
foo f x = do
    v <- x
    f v
```

Remember, `return` is not a return statement like in other languages – it does not signal the end of control flow..

Do Notation

```
foo f x y = x >>= (\_ -> y >>= (\v -> return (f v)))
```

```
foo f x = do
    _ <- x
    v <- y
    return (f v)
```

Do Notation

```
foo f x y = x >>= (\_ -> y >>= (\v -> return (f v)))
```

```
foo f x = do
    x
    v <- y
    return (f v)
```

Intermediate lines without a `<-` throw away their result.

Do Notation

```
foo f x y = x >> (y >>= (\v -> return (f v)))
```

```
foo f x = do
    x
    v <- y
    return (f v)
```

Intermediate lines without a `<-` throw away their result.

Do Notation

```
divAdd’ x1 x2 x3 x4 =
    safeDiv x1 x2 >>=
    \r1 -> (safeDiv x3 x4 >>= \r2 -> return (r1 + r2)
```

Do Notation

```
divAdd’ x1 x2 x3 x4 =
    safeDiv x1 x2 >>=
    \r1 -> (safeDiv x3 x4 >>= \r2 -> return (r1 + r2)

divAdd’’ x1 x2 x3 x4 = do
    r1 <- safeDiv x1 x2
    r2 <- safeDiv x3 x4
    return (r1 + r2)
```

Do Notation

```
divAdd' x1 x2 x3 x4 =
    safeDiv x1 x2 >>=
    \r1 -> (safeDiv x3 x4 >>= \r2 -> return (r1 + r2)

divAdd'' x1 x2 x3 x4 = do
    r1 <- safeDiv x1 x2
    r2 <- safeDiv x3 x4
    return (r1 + r2)
```

This is designed to look like familiar imperative code, but it is still functional!

Do Notation

```
divAdd' x1 x2 x3 x4 =
    safeDiv x1 x2 >>=
    \r1 -> (safeDiv x3 x4 >>= \r2 -> return (r1 + r2)

divAdd'' x1 x2 x3 x4 = do
    r1 <- safeDiv x1 x2
    r2 <- safeDiv x3 x4
    return (r1 + r2)
```

do notation compiles to regular code that uses >>=, so these representations are the same

Do Notation

```
divAdd' x1 x2 x3 x4 =
    safeDiv x1 x2 >>=
    \r1 -> (safeDiv x3 x4 >>= \r2 -> return (r1 + r2)

divAdd'' x1 x2 x3 x4 = do
    r1 <- safeDiv x1 x2
    r2 <- safeDiv x3 x4
    return (r1 + r2)
```

We cannot mix which monad we are compiling our `do` block to!

Do Notation

```
foo = do
    x <- [1,2,3]
    y <- Just 4
    return (x + y)
```

Do Notation

```
foo = do
    x <- [1,2,3]
    y <- Just 4
    return (x + y)
```

Do Notation

```
foo = [1,2,3] >>= (\x -> Just 4 >>= (\y -> return (x + y)))
```

Do Notation

```
[Int]
foo = [1,2,3] >>= (\x -> Just 4 >>= (\y -> return (x + y)))
```

Do Notation

```
[Int] -> (Int -> [b]) -> [b]
```

```
foo = [1,2,3] >>= (\x -> Just 4 >>= (\y -> return (x + y)))
```

Do Notation

```
Int -> [b]
```

```
foo = [1,2,3] >>= (\x -> Just 4 >>= (\y -> return (x + y)))
```

Do Notation

```
[b]

foo = [1,2,3] >>= (\x -> Just 4 >>= (\y -> return (x + y)))
```

Do Notation

```
[a] -> (a -> [b]) -> [b]
```

```
foo = [1,2,3] >>= (\x -> Just 4 >>= (\y -> return (x + y)))
```

Do Notation

```
[a]

foo = [1,2,3] >>= (\x -> Just 4 >>= (\y -> return (x + y)))
```

Do Notation

```
foo = do
    x <- [1,2,3]
    y <- Just 4
    return (x + y)
```

If one of the subcomputations we are combining can fail, the whole computation must be able to fail.

Do Notation

```
foo = do
    x <- [1,2,3]
    y <- [Just 4]
    return (x + y)
```

We can have other monads in our `do` block, we just cannot mix which bind `<-` refers to.

Do Notation

```
divAdd' x1 x2 x3 x4 =
    safeDiv x1 x2 >>=
    \r1 -> (safeDiv x3 x4 >>= \r2 -> return (r1 + r2)

divAdd'' x1 x2 x3 x4 = do
    r1 <- safeDiv x1 x2
    r2 <- safeDiv x3 x4
    return (r1 + r2)
```

We cannot mix which monad we are compiling our `do` block to!

Maybe

```
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x : _) = Just x

addFirsts :: [Int] -> [Int] -> Maybe Int
addFirsts xs ys = case safeHead xs of
    Nothing -> Nothing
    Just x -> case safeHead ys of
        Nothing -> Nothing
        Just y -> Just (x + y)
```

Maybe

```
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x : _) = Just x

addFirsts :: [Int] -> [Int] -> Maybe Int
addFirsts xs ys = case safeHead xs of
    Nothing -> Nothing
    Just x -> case safeHead ys of
        Nothing -> Nothing
        Just y -> Just (x + y)
```

Rewrite `addFirsts` using `do` notation.

Maybe

```
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x : _) = Just x

addFirsts :: [Int] -> [Int] -> Maybe Int
addFirsts xs ys = do
    x <- safeHead xs
    y <- safeHead ys
    return (x + y)
```

Maybe

```
addFirsts :: [Int] -> [Int] -> Maybe Int
addFirsts xs ys = do
    x <- safeHead xs
    y <- safeHead ys
    return (x + y)
```

```
addFirsts [1,2] [] = ?
```

Maybe

```
addFirsts :: [Int] -> [Int] -> Maybe Int
addFirsts xs ys = do
    x <- safeHead xs
    y <- safeHead ys
    return (x + y)


addFirsts [1,2] [3] = ?
```

Maybe

```
addFirsts :: [Int] -> [Int] -> Maybe Int
addFirsts xs ys = do
    x <- safeHead xs
    y <- safeHead ys
    return (x + y)


addFirsts [1,2] [3,4] = ?
```

Monads

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Monads model *effects*.

What other effects can we model?

Monads

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Monads model *effects*.

What other effects can we model?

- State
- IO
- Nondeterminism

Monads

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Monads model *effects*.

What other effects can we model?

- State
- IO
- **Nondeterminism**

List

Lists can be thought of as a model for *nondeterminism*.

```
f(x) = sample(0,x)
```

2

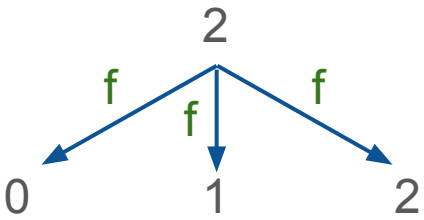
```
[2]
```

```
g(x) = if coinFlip()
       then x
       else x + 1

h(x) = 2 * x
```

List

Lists can be thought of as a model for *nondeterminism*.

f(x) = sample(0,x)



[2]

g(x) = if coinFlip()
then x
else x + 1

[0,1,2]

h(x) = 2 * x

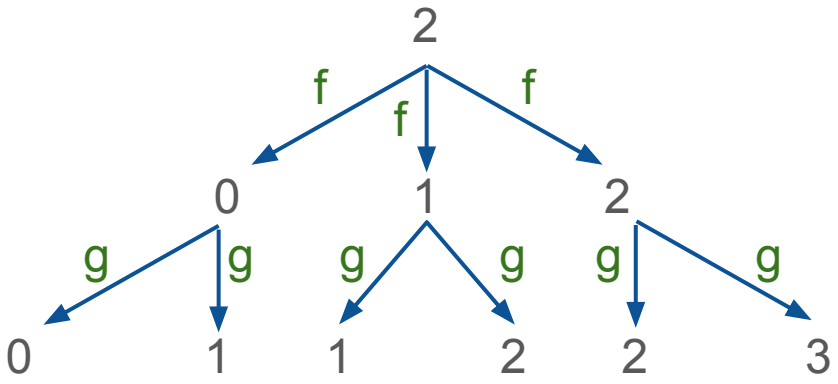
List

Lists can be thought of as a model for *nondeterminism*.

f(x) = sample(0,x)

g(x) = if coinFlip()
then x
else x + 1

h(x) = 2 * x



[2]

[0,1,2]

[0,1,1,2,2,3]

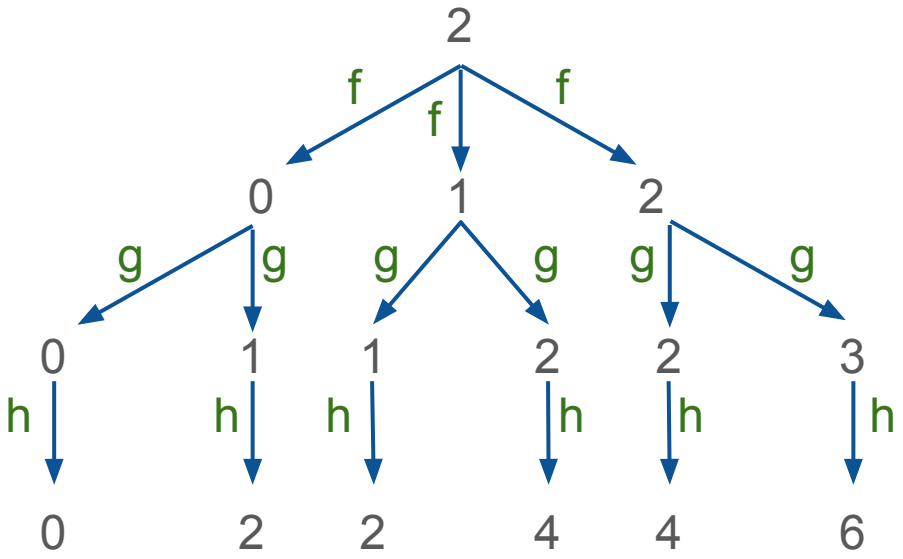
List

Lists can be thought of as a model for *nondeterminism*.

f(x) = sample(0,x)

g(x) = if coinFlip()
then x
else x + 1

h(x) = 2 * x



[2]

[0,1,2]

[0,1,1,2,2,3]

[0,2,2,4,4,6]

List

Lists can be thought of as a model for *nondeterminism*.

```
f(x) = sample(0,x)

g(x) = if coinFlip()
       then x
       else x + 1

h(x) = 2 * x
```

List

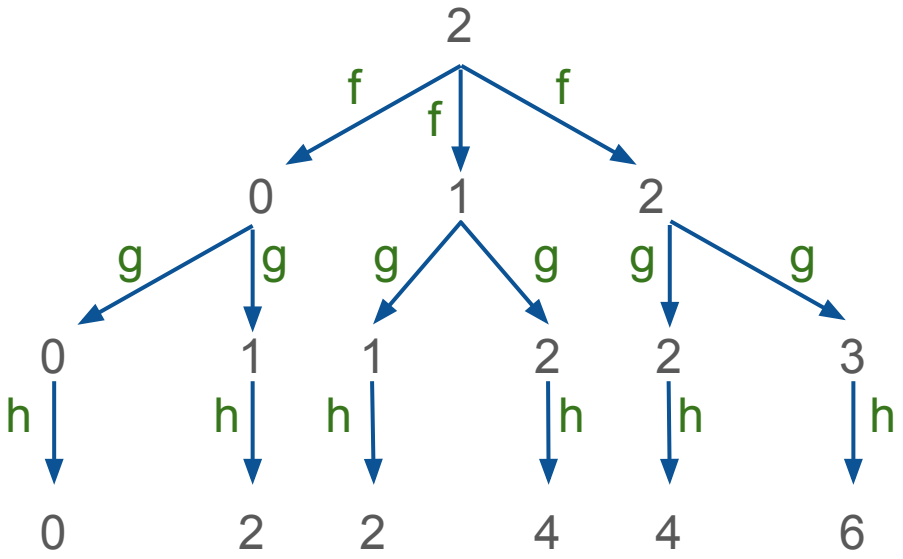
Lists can be thought of as a model for *nondeterminism*.

```
f x = [0..x]

g x = [x, x+1]

h x = [2 * x]
```

List

Lists can be thought of as a model for *nondeterminism*.

```
f x = [0..x]

g x = [x, x+1]

h x = [2 * x]
```




```
[2]

[0,1,2]

[0,1,1,2,2,3]

[0,2,2,4,4,6]
```

List

Lists can be thought of as a model for *nondeterminism*.

```haskell
f :: Int -> [Int]
f x = [0..x]

g :: Int -> [Int]
g x = [x, x+1]

h :: Int -> [Int]
h x = [2 * x]
```

```haskell
[2]

?? f [2] = [0,1,2]

?? g [0,1,2] = [0,1,1,2,2,3]

?? h [0,1,1,2,2,3] = [0,2,2,4,4,6]
```

List

Lists can be thought of as a model for *nondeterminism*.

```haskell
f :: Int -> [Int]
f x = [0..x]

g :: Int -> [Int]
g x = [x, x+1]

h :: Int -> [Int]
h x = [2 * x]
```

```haskell
[2]

concatMap f [2] = [0,1,2]

concatMap g [0,1,2] = [0,1,1,2,2,3]

concatMap h [0,1,1,2,2,3] = [0,2,2,4,4,6]
```

List

Lists can be thought of as a model for *nondeterminism*.

`concatMap` sequences nondeterminism.

It takes a list of possible values and a nondeterministic function and shows all values possible after applying the function (with multiplicity).

List

```
instance  Monad []  where
    (>>=) :: [a] -> (a -> [b]) -> [b]
    xs >>= f    = concatMap f xs
```

List

```
instance  Monad []  where
    (>>=) :: [a] -> (a -> [b]) -> [b]
    xs >>= f    = concatMap f xs

    return :: a -> [a]
    return x = ??
```

List

```
instance  Monad []  where
    (>>=) :: [a] -> (a -> [b]) -> [b]
    xs >>= f    = concatMap f xs

    return :: a -> [a]
    return x = [x]
```

List

```
f x = [0..x]
g x = [x, x+1]
h x = [2 * x]

foo x = do
    y <- f x
    z <- g y
    h z
```

List

```
f x = [0..x]
g x = [x, x+1]
h x = [2 * x]

foo x = do            2
    y <- f x      [0,1,2]
    z <- g y      [0,1,1,2,2,3]
    h z               [0,2,2,4,4,6]
```

List

```
f x = [0..x]
g x = [x, x+1]
h x = [2 * x]

foo x = do            2
    y <- f x      [0,1,2]
    z <- g y      [0,1,1,2,2,3]
    h z               [0,2,2,4,4,6]
```

What is the type of `y`?

List

```
f x = [0..x]
g x = [x, x+1]
h x = [2 * x]

foo x = do            2
    y <- f x      [0,1,2]
    z <- g y      [0,1,1,2,2,3]
    h z               [0,2,2,4,4,6]
```

`Int`! `f x` is a nondeterministic computation, and `<-` handles all the nondeterminism so we can work with it as though it just produces one value.

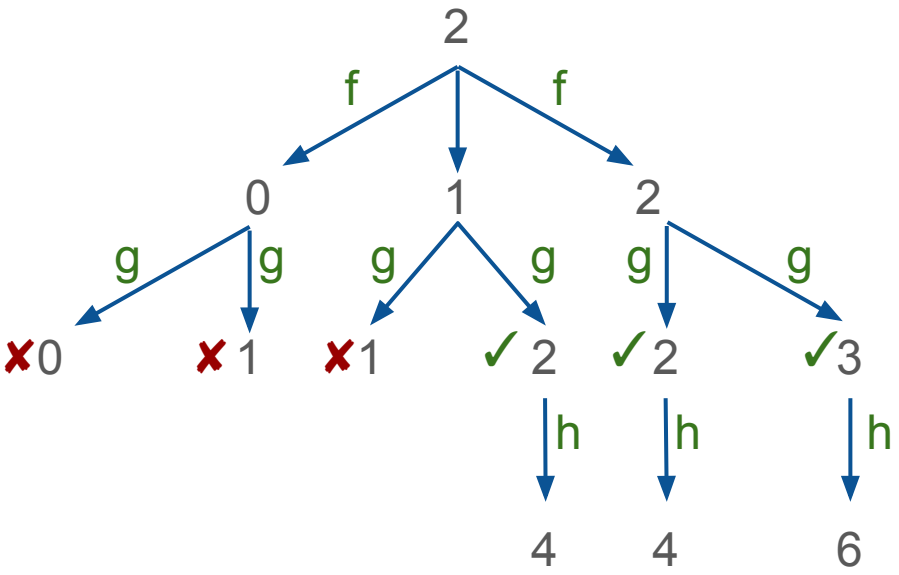
Guard

```
f x = [0..x]
g x = [x, x+1]
h x = [2 * x]

foo x = do              2
    y <- f x      [0,1,2]
    z <- g y      [0,1,1,2,2,3]
    guard (z > 1)     [2,2,3]
    h z               [4,4,6]
```

Along each control-flow path, `guard` either ends computation or lets it continue.

Guard

```
f x = [0..x]
g x = [x, x+1]
h x = [2 * x]

foo x = do              2
    y <- f x        [0,1,2]
    z <- g y        [0,1,1,2,2,3]
    guard (z > 1)     [2,2,3]
    h z               [4,4,6]
```



Along each control-flow path, `guard` either ends computation or lets it continue.

Guard

```
addFirstsEven xs ys = do
    x <- safeHead xs
    y <- safeHead ys
    guard (even x)
    return (x + y)
```

Guard

```
addFirstsEven xs ys =
```

List

```
foo = [ x `div` 2 | x <- [0..10], even x]

foo = do
    x <- [0..10]
    guard (even x)
    return (x `div` 2)

foo = [0..10] >>= \x -> guard (even x) >> return (x `div` 2)
```

List

```
foo = [ x `div` 2 | x <- [0..10], even x]

foo = do
    x <- [0..10]
    guard (even x)
    return (x `div` 2)

foo = [0..10] >>= \x -> guard (even x) >> return (x `div` 2)
```

What does `foo` return?

List

```
foo = [ x `div` 2 | x <- [0..10], even x]

foo = do
    x <- [0..10]
    guard (even x)
    return (x `div` 2)

foo = [0..10] >>= \x -> guard (even x) >> return (x `div` 2)
```

What does `foo` return? `[0,1,2,3,4,5]`

List

```
sequence :: Monad m => [m a] -> m [a]
sequence [] = return []
sequence (mx : mxs) =
  mx >>= (\y -> (sequence mxs >>= (\ys -> return (y : ys))))
```

Note: `>>=` is right-associative, so these parentheses are redundant.

List

```
sequence :: Monad m => [m a] -> m [a]
sequence [] = return []
sequence (mx : mxs) =
  mx >>= (\y -> (sequence mxs >>= (\ys -> return (y : ys))))
```

For `Maybe`, this means the whole list fails if any element failed.
For `List`, this is all the ways to choose one value from each list:

```
sequence [“abc”, “de”] = ["ad","ae","bd","be","cd","ce"]
sequence [“abc”, “de”, “”] = []
```

Functor vs. Applicative vs. Monad

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b


class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b


class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Functor vs. Applicative vs. Monad

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b


class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b


class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Functor vs. Applicative vs. Monad

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

We can modify values in an effectful setting.

```
class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Functor vs. Applicative vs. Monad

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

We can modify values in an effectful setting.

```
class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

We can run effectful computations in parallel and combine their results.

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

Functor vs. Applicative vs. Monad

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

We can modify values in an effectful setting.

```
class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

We can run effectful computations in parallel and combine their results.

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

We can sequence effects, with later control flow depending on earlier results.

Functor vs. Applicative vs. Monad

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

We can modify values but not change the effect.

```
class Functor f => Applicative f where
    pure :: a -> f a
    (<*>) :: f (a -> b) -> f a -> f b
```

We can combine effectful computations, but the effect cannot depend on their results.

```
class Applicative m => Monad m where
    return  :: a -> m a
    (>>=)   :: m a -> (a -> m b) -> m b
```

We can sequence effects, with later effects depending on the results of former computations.